

Bash Mastery — Quizzes

bash-learning curriculum

2026-06-04

Table of Contents

1. Module 1 — Bash Basics
2. Module 2 — Conditionals
3. Module 3 — Loops
4. Module 4 — Functions
5. Module 5 — I/O & Redirection
6. Module 6 — Arrays
7. Module 7 — String Manipulation & Regex
8. Module 8 — Processes, Signals, Traps
9. Module 9 — Text Processing (grep / sed / awk)
10. Module 10 — Error Handling & Debugging
11. Module 11 — Advanced Patterns

Module 1 — Bash Basics

Quiz: Bash Basics

Multiple-choice

1. What is the main purpose of the shebang line `#!/bin/bash`?
 - A. It adds comments to the file.
 - B. It tells the system which interpreter should run the script.
 - C. It makes every variable global.
 - D. It prints Bash version information.
2. Which variable assignment is correct in Bash?
 - A. `name = "Sai"`
 - B. `$name="Sai"`
 - C. `name="Sai"`
 - D. `name := "Sai"`
3. Why does Bash require no spaces around `=` in variable assignment?
 - A. Because spaces turn the line into command + arguments instead of assignment.
 - B. Because spaces are only allowed inside loops.
 - C. Because quotes stop working otherwise.
 - D. Because `=` only works in comments.
4. When is `${name}` better than `$name`?
 - A. Only inside comments.
 - B. When the variable is next to other text and Bash must know where the name ends.
 - C. Only for numbers.
 - D. Only after read.
5. In a script, what does `$0` mean?

- A. The first user argument.
 - B. The number of arguments.
 - C. The script name or path used to run the script.
 - D. All arguments as one list.
6. In a script, what does \$# mean?
- A. The count of command-line arguments.
 - B. The value of the last argument.
 - C. The current line number.
 - D. The shebang path.
7. What does "\$*" usually represent?
- A. Only the script name.
 - B. All command-line arguments combined into one string.
 - C. The first and second arguments only.
 - D. The number of arguments.
8. Which line correctly reads user input into a variable called name?
- A. read \$name
 - B. read name
 - C. read =name
 - D. read "\$name"
9. What is the problem with read \$name?
- A. It always reads two words.
 - B. It expands \$name first instead of using name as the target variable.
 - C. It only works in loops.
 - D. It clears every other variable.
10. Which command shows a prompt and stores input in city on one line?
- A. read city -p "Enter city: "
 - B. read -prompt "Enter city: " city
 - C. read -p "Enter city: " city
 - D. read "Enter city: " city

Fill in the blank

11. The shortcut that stores the first command-line argument is ____.
12. The shortcut that stores all command-line arguments is ____.
13. Complete the safer expansion form: Hello, ____!

Short answer: What does this script print?

14.

```
#!/bin/bash
name="Hitesh"
echo "Hello $name"
```
15.

```
#!/bin/bash
echo "Count: $#"
```

If the script is run as ./count.sh red blue green, what is printed?

Answer Key

1. B
2. C
3. A
4. B
5. C
6. A
7. B
8. B
9. B
10. C
11. \$1
12. \$@
13. \${name}

14. Hello Hitesh
15. Count: 3

Module 2 — Conditionals

Quiz: Bash Conditionals

Multiple-choice

1. What is the main difference between [...] and [[...]] in Bash?
 - A. [...] can use regex, but [[...]] cannot.
 - B. [[...]] is Bash-specific and supports safer pattern/logic features.
 - C. [...] only works for numbers.
 - D. [[...]] only works inside loops.
2. Why should variables usually be quoted inside [...]?
 - A. To make them uppercase automatically.
 - B. To avoid word splitting and pathname expansion problems.
 - C. To force numeric comparison.
 - D. To convert empty strings to zero.
3. Which test checks whether a and b are numerically equal?
 - A. ["\$a" = "\$b"]
 - B. ["\$a" == "\$b"]
 - C. ["\$a" -eq "\$b"]
 - D. [["\$a" =~ "\$b"]]
4. Which expression does a lexicographic (alphabetical) comparison correctly?
 - A. ["\$x" -lt "\$y"]
 - B. [["\$x" < "\$y"]]
 - C. ["\$x" =< "\$y"]
 - D. [["\$x" -eq "\$y"]]
5. What does -z check?
 - A. The file size is zero.
 - B. The variable contains only digits.
 - C. The string is empty.
 - D. The string is quoted.
6. What does -n check?
 - A. The string is not empty.
 - B. The value is negative.
 - C. The variable is numeric.
 - D. The file is new.
7. Which option correctly matches the file test flags?
 - A. -f directory, -d regular file, -e executable
 - B. -f regular file, -d directory, -e path exists
 - C. -f full path, -d device, -e empty file
 - D. -f found, -d deleted, -e enabled
8. Which mapping is correct?
 - A. -r readable, -w writable, -x executable, -s non-empty file, -L symbolic link
 - B. -r renamed, -w whole file, -x extra file, -s safe file, -L large file
 - C. -r regular file, -w working directory, -x hidden file, -s symlink, -L locked file
 - D. -r root-only, -w wildcard, -x empty, -s source file, -L local file
9. In a case statement, what is the difference between ;;, ;&, and ;;&?
 - A. They are three ways to end the script.
 - B. ;; stops, ;& runs the next branch body, ;;& re-tests the next pattern list.
 - C. ;& means default case, ;;& means exit, ;; means continue loop.
 - D. There is no difference.
10. How do you define the default branch in a case statement?
 - A. default)
 - B. else)
 - C. *)
 - D. ?)

Fill in the blank

11. The file test flag for “path exists” is ____.
12. The file test flag for “symbolic link” is ____.
13. Complete the pattern used for the default case branch: ____).

Predict the output

14.

```
#!/bin/bash
name=""
if [ -z "$name" ]; then
    echo "empty"
else
    echo "filled"
fi
```

What is printed?

15.

```
#!/bin/bash
value="report.txt"
if [[ $value == *.txt ]]; then
    echo "text file"
else
    echo "other"
fi
```

What is printed?

Answer Key

1. B
2. B
3. C
4. B
5. C
6. A
7. B
8. A
9. B
10. C
11. -e
12. -L
13. *
14. empty
15. text file

Module 3 — Loops

Module 03 — Loops Quiz

Multiple Choice (10)

1. Which loop is best when you want to run with a numeric counter from 1 to 10?
 - A. for item in list
 - B. for ((i=1; i<=10; i++))
 - C. until read x
 - D. case ... esac
2. What is the main difference between while and until?
 - A. while runs while the condition is true; until runs while the condition is false

- B. while is only for files; until is only for numbers
 - C. until is faster than while
 - D. There is no difference
3. What does continue do inside a loop?
 - A. Ends the script
 - B. Exits the loop completely
 - C. Skips the rest of the current iteration
 - D. Repeats the previous iteration
 4. What does break do inside a loop?
 - A. Skips one command
 - B. Exits the current loop immediately
 - C. Restarts the loop
 - D. Closes the terminal
 5. Which is the safest canonical pattern for reading a file line by line?
 - A. for line in \$(cat file)
 - B. cat file | while read line; do ...; done
 - C. while IFS= read -r line; do ...; done < file
 - D. read file line
 6. Why is for f in \$(ls) a bad idea?
 - A. ls cannot list files
 - B. It can split filenames on spaces and other whitespace
 - C. It only works for hidden files
 - D. It is slower than echo
 7. What does read -r line protect against?
 - A. Arithmetic expansion
 - B. Command substitution
 - C. Backslash interpretation by read
 - D. Filename globbing
 8. Which statement about IFS is true?
 - A. It controls how Bash splits input into fields
 - B. It is only used by echo
 - C. It permanently disables whitespace
 - D. It creates arrays automatically
 9. Which loop can easily become infinite if the condition never changes?
 - A. while
 - B. until
 - C. Both while and until
 - D. Neither
 10. Which is the best way to loop over array elements that may contain spaces?
 - A. for x in \${arr[@]}
 - B. for x in \$(printf '%s\n' "\${arr[@]}")
 - C. for x in "\${arr[@]}"
 - D. for x in \$arr

Fill in the Blank (3)

11. The loop that runs until a condition becomes true is called an _____ loop.
12. To read raw lines safely from a file, the usual pattern starts with while IFS= read -r _____.
13. To stop a loop as soon as a match is found, use the _____ command.

Predict the Output (2)

14. What is the output?

```
for ((i=1; i<=4; i++))
do
  if [ "$i" -eq 3 ]
  then
    continue
```

```

fi
echo "$i"
done

```

15. What is the output?

```

count=1
until [ "$count" -gt 3 ]
do
    echo "$count"
    ((count++))
done

```

Answer Key

1. **B**
2. **A**
3. **C**
4. **B**
5. **C**
6. **B**
7. **C**
8. **A**
9. **C**
10. **C**
11. **until**
12. **line**
13. **break**
- 14.

```

1
2
4

```

15.

```

1
2
3

```

Module 4 — Functions

Module 04: Functions — Quiz

Questions

1. **MCQ:** Which syntax is the most common portable way to define a bash function?
 - A. `def name():`
 - B. `name() { ... }`
 - C. `func name => { ... }`
 - D. `function: name`
2. **Fill in the blank:** Inside a function, _____ refers to the first argument passed to that function.
3. **MCQ:** Inside a function, what does `$0` refer to?
 - A. The function name
 - B. The first function argument
 - C. The script name
 - D. The last command status

4. **Predict output:**

```
show() {  
    echo "$#:$1:$2"  
}  
show cat dog
```

5. **MCQ:** What does return 7 do inside a bash function?

- A. Prints 7
- B. Sets the function exit status to 7
- C. Stores 7 in \$1
- D. Sends 7 to stdout and stderr

6. **Fill in the blank:** To capture text printed by a function, use result=\$(_____).

7. **MCQ:** Which statement about return in bash is correct?

- A. It can return any size integer safely
- B. It is mainly for exit status values
- C. It returns strings directly
- D. It is the same as echo

8. **Predict output:**

```
greet() {  
    local name="${1:-friend}"  
    echo "Hi, $name"  
}  
greet
```

9. **MCQ:** Why is local useful inside a function?

- A. It makes the variable available to all scripts
- B. It prevents the function from changing outer variables accidentally
- C. It automatically exports the variable
- D. It converts the variable into an array

10. **Fill in the blank:** The usual exit-status range for return is ___ to ___.

11. **MCQ:** Why is the function keyword considered less portable than name() { ... }?

- A. It only works in Python
- B. It is bash/ksh-style and not required by POSIX sh
- C. It requires root privileges
- D. It only works for recursive functions

12. **Predict output:**

```
value="outside"  
demo() {  
    local value="inside"  
    echo "$value"  
}  
demo  
echo "$value"
```

13. **MCQ:** Which is the best way for a function to provide a computed string like full_name?

- A. return "full_name"
- B. echo "full_name" and capture it with \$(...)
- C. exit "full_name"
- D. Put it in \$0

14. **Fill in the blank:** "\$@" expands to _____.

15. **Predict output:**

```

add() {
    echo $(( $1 + $2 ))
}
total="$(add 4 9)"
echo "$total"

```

Answer Key

1. **B** — name() { ... }
2. **\$1**
3. **C** — the script name
4. **2:cat:dog**
5. **B** — sets the function exit status to 7
6. **the function call**, for example myfunc arg1 arg2
7. **B** — it is mainly for exit status values
8. **Hi, friend**
9. **B** — it prevents accidental changes to outer variables
10. **0 to 255**
11. **B** — it is bash/ksh-style and not required by POSIX sh
12. Output is: text inside outside
13. **B** — echo plus command substitution
14. **all positional arguments as separate words when quoted**
15. **13**

Module 5 — I/O & Redirection

Module 05 Quiz: I/O and Redirection

Multiple Choice (10)

1. Which file descriptor number is **stdin**?
 - A. 0
 - B. 1
 - C. 2
 - D. 3
2. Which operator appends stdout to a file instead of overwriting it?
 - A. >
 - B. >>
 - C. 2>
 - D. |
3. What does 2> errors.log do?
 - A. Redirect stdout to errors.log
 - B. Redirect stdin from errors.log
 - C. Redirect stderr to errors.log
 - D. Merge stdout and stderr
4. Which command sends both stdout and stderr to the same file **in the correct order**?
 - A. cmd 2>&1 > out.log
 - B. cmd > out.log 2>&1
 - C. cmd < out.log 2>&1
 - D. cmd | out.log 2>&1
5. What is a here-string?
 - A. Multi-line inline input using <<EOF
 - B. A single string passed to stdin using <<<
 - C. A way to append to a file with >>
 - D. A way to duplicate output with tee
6. What does < <(cmd) mean?
 - A. Append command output to a file
 - B. Read stdin from the output of cmd using process substitution
 - C. Run cmd in the background
 - D. Merge stderr into stdout
7. What is the main purpose of tee?

- A. Convert stderr into stdout
 - B. Read a file into stdin
 - C. Write output to both stdout and a file
 - D. Sort command output before diffing
8. In bash, `diff <(sort a.txt) <(sort b.txt)` is useful because it:
- A. Deletes both files after sorting
 - B. Compares sorted command outputs without creating sorted temp files manually
 - C. Redirects stderr to diff
 - D. Forces both files to have the same content
9. In a pipeline like `printf 'a\nb\n' | while read -r line; do count=$((count+1)); done`, why is `count` often unchanged afterward?
- A. `read` cannot handle piped input
 - B. `printf` resets shell variables
 - C. The loop may run in a subshell created by the pipe
 - D. `count` is a reserved variable name
10. Which redirection discards both stdout and stderr?
- A. `2> /dev/null`
 - B. `> /dev/null`
 - C. `>/dev/null 2>&1`
 - D. `<< /dev/null`

Fill in the Blank (3)

11. The standard error file descriptor number is ____.
12. A multi-line block of inline input introduced with `<<EOF` is called a ____.
13. To append stdout to `log.txt`, use ____ `log.txt`.

Predict the Output (2)

14. Predict the terminal output:

```
{
  echo "OUT"
  echo "ERR" >&2
} > result.log 2>&1
cat result.log
```

15. Predict the terminal output:

```
count=0
printf 'x\ny\n' | while read -r line; do
  count=$((count+1))
  echo "inside:$count"
done
echo "outside:$count"
```

Answer Key

Multiple Choice

1. A
2. B
3. C
4. B
5. B
6. B
7. C
8. B
9. C

10. C

Fill in the Blank

11. 2

12. here-document

13. >>

Predict the Output

14.

```
OUT
ERR
```

Explanation: both streams went into result.log, and then cat result.log printed the file contents.

15.

```
inside:1
inside:2
outside:0
```

Explanation: the while loop runs in a subshell in many bash pipeline cases, so the outer count variable does not keep the updated value.

Order Matters Note

- `cmd >file 2>&1` sends both stdout and stderr to file.
- `cmd 2>&1 >file` usually leaves stderr pointing at the old stdout destination (often the terminal), while stdout goes to file.

Module 6 — Arrays

Module 06 Quiz: Bash Arrays

Multiple Choice (10)

Choose the best answer.

1. Which syntax creates an indexed array with three elements?
 - A. `declare -A arr=(a b c)`
 - B. `arr=(a b c)`
 - C. `arr={a,b,c}`
 - D. `arr=[a b c]`
2. What does `${#arr[@]}` return?
 - A. The length of the first string element
 - B. The highest numeric index in the array
 - C. The number of assigned elements in the array
 - D. The number of characters in the array name
3. With `arr=("a b" "c")`, what does `${arr[@]}` preserve?
 - A. It joins everything into one string
 - B. It preserves each element as a separate word
 - C. It removes spaces inside elements
 - D. It sorts the array before expansion
4. With `arr=("a b" "c")`, what does `${arr[*]}` produce?
 - A. Two separate words
 - B. Three separate words
 - C. One single word containing all elements joined by the first character of IFS

- D. An error
- 5. What happens after unset 'arr[1]' on an indexed array?
 - A. All later elements shift left automatically
 - B. The array becomes empty
 - C. A gap remains; the array can become sparse
 - D. The array turns into an associative array
- 6. Which syntax lists the assigned indices of an indexed array?
 - A. \${arr[#]}
 - B. \${!arr[@]}
 - C. \${arr[@]!}
 - D. \${#arr[*]}
- 7. Which declaration is required for associative arrays?
 - A. declare -i map
 - B. declare -r map
 - C. declare -x map
 - D. declare -A map
- 8. Which loop is safest for iterating through array elements that may contain spaces?
 - A. for x in \${arr[*]}; do ...; done
 - B. for x in \${arr[@]}; do ...; done
 - C. for x in "\${arr[@]}"; do ...; done
 - D. for x in \$arr; do ...; done
- 9. What does declare -n ref=\$1 do inside a function?
 - A. Creates a numeric variable
 - B. Creates a nameref pointing to another variable by name
 - C. Exports a variable to child processes
 - D. Makes a copy of an array
- 10. Which command splits a string into an array using shell word splitting rules?
 - A. read -ra arr <<< "\$line"
 - B. printf -ra arr "\$line"
 - C. echo -ra arr "\$line"
 - D. set -A arr "\$line"

Fill in the blanks (3)

11. The expansion used to get all keys from an associative array named map is _____.
12. To append two elements e and f to an indexed array named arr, write _____.
13. To get a slice starting at index 2 with count 3 from arr, write _____.

Predict the output (2)

14. Predict the output:

```
arr=("a b" "c")
printf '<%s>\n' "${arr[@]}"
```

15. Predict the output:

```
arr=(zero one two)
unset 'arr[1]'
echo "indices=${!arr[@]} values=${arr[*]} length=${#arr[@]}"
```

Answer Key

Multiple Choice

1. **B**

2. **C**
3. **B**
4. **C**
5. **C**
6. **B**
7. **D**
8. **C**
9. **B**
10. **A**

Fill in the blanks

11. `${!map[@]}`
12. `arr+=(e f)`
13. `${arr[@]:2:3}`

Predict the output

14.

```
<a b>
<c>
```

15.

```
indices=0 2 values=zero two length=2
```

Why: unset 'arr[1]' removes only index 1. The array becomes sparse, so indices 0 and 2 remain assigned.

Module 7 — String Manipulation & Regex

Module 07 Quiz: String Manipulation and Regex

Multiple Choice (1-10)

1. What does `${path##*/}` usually return? - A. Everything before the first slash - B. Everything after the last slash - C. Everything after the first slash - D. Everything before the last slash
2. Which expression removes the shortest matching suffix? - A. `${file%*.}` - B. `${file##*.}` - C. `${file%.*}` - D. `${file#*.}`
3. What is the difference between `${text/foo/bar}` and `${text//foo/bar}`? - A. No difference - B. The first removes text, the second replaces text - C. The first replaces the first match, the second replaces all matches - D. The first uses regex, the second uses glob
4. Which operator both supplies a fallback and assigns it back to the variable? - A. `${var:-fallback}` - B. `${var:=fallback}` - C. `${var:?fallback}` - D. `${var:+fallback}`
5. What does `${var:+yes}` expand to when var is set to hello? - A. hello - B. yes - C. an empty string - D. an error
6. In `[[ $value =~ $regex ]]`, where do captured groups go after a successful match? - A. `$MATCHES` - B. `${REGEX_GROUPS[@]}` - C. `${BASH_REMATCH[@]}` - D. `$?`

7. Which statement about `[[ $text =~ regex ]]` is correct? - A. The right side should always be quoted - B. The right side should not be quoted if you want regex matching - C. The left side must be unquoted and unbraced - D. `==` works only in `/bin/sh`
8. Which `printf` format prints an integer padded with leading zeroes to width 5? - A. `%5s` - B. `%05d` - C. `%.5d` - D. `%q`
9. Which expansion uppercases all letters in a variable on bash 4+? - A. `${name^}` - B. `${name,}` - C. `${name^^}` - D. `${name~~}`
10. Which statement is true about `#` vs `##` and `%` vs `%%`? - A. `#` and `%` are greedy; `##` and `%%` are non-greedy - B. `#` and `%` work only with regex - C. `#/%` remove the shortest match; `##/%%` remove the longest match - D. They are only for arrays

Fill in the Blanks (11-13)

11. Complete the expression that prints the last 3 characters of word:
`${word: ____ }`
12. Complete the operator that uses `guest` when `name` is unset or empty, but does **not** assign it: `${name____guest}`
13. Complete the `printf` format for shell-escaped output: `printf 'quoted: ____\n' "$value"`

Predict the Output (14-15)

14. Predict the output:

```
file="archive.tar.gz"
echo "${file%.*}"
echo "${file%%.*}"
```

15. Predict the output:

```
text="one two two"
echo "${text/two/2}"
echo "${text//two/2}"
```

Answer Key

1. **B**
2. **C**
3. **C**
4. **B**
5. **B**
6. **C**
7. **B**
8. **B**
9. **C**
10. **C**
11. **-3**
12. **:-**
13. **%q**
- 14.

```
archive.tar
archive
```

- 15.

```
one 2 two
```

Module 8 — Processes, Signals, Traps

Module 08 Quiz — Processes, Signals, and Traps

Multiple Choice (1-10)

1. Which variable stores the PID of the current shell?
 - A. \$PPID
 - B. \$\$
 - C. \$!
 - D. \$?
2. Which variable stores the PID of the most recently started background job?
 - A. \$!
 - B. \$\$
 - C. \$#
 - D. \$0
3. What does \$PPID represent?
 - A. The PID of the current script file
 - B. The PID of the previous command
 - C. The PID of the parent process
 - D. The PID of the process group leader
4. Which signal is normally sent by pressing Ctrl-C?
 - A. SIGTERM
 - B. SIGHUP
 - C. SIGKILL
 - D. SIGINT
5. Which signal is the usual polite request to terminate a process?
 - A. SIGKILL
 - B. SIGTERM
 - C. SIGUSR1
 - D. SIGSTOP
6. Why can SIGKILL not be trapped?
 - A. Because only root can send it
 - B. Because bash disables traps for signal 9
 - C. Because the kernel enforces immediate termination
 - D. Because it is only valid for background jobs
7. What is the main difference between (...) and { ...; }?
 - A. Parentheses run faster than braces
 - B. Parentheses create a subshell; braces run in the current shell
 - C. Braces create a subshell; parentheses run in the current shell
 - D. There is no difference
8. What does trap 'cleanup' EXIT do?
 - A. Calls cleanup only on syntax errors
 - B. Calls cleanup when the shell exits
 - C. Calls cleanup before every command
 - D. Calls cleanup only when a background job finishes
9. What does wait with no arguments do?
 - A. Waits for the last foreground command only
 - B. Waits for a keypress
 - C. Waits for all current background jobs in the shell
 - D. Waits only for jobs started with nohup
10. What is the purpose of timeout 5 some_command?
 - A. Delay starting the command for 5 seconds
 - B. Repeat the command every 5 seconds
 - C. Kill the command immediately
 - D. Stop the command if it runs longer than 5 seconds

Fill in the Blank (11-13)

11. The variable ____ stores the PID of the most recently started background process.
12. The signal number 15 usually refers to ____.
13. In bash, trap 'cleanup' ____ is the common pattern for guaranteed cleanup at script exit.

Predict the Output / Behavior (14-15)

14. Predict the final value of name after this script runs:

```
name="outer"
(
  name="inner"
)
echo "$name"
```

15. Predict the final value of count after this script runs:

```
count=1
{
  count=5
}
echo "$count"
```

Answer Key

1. **B** — \$\$
2. **A** — \$!
3. **C** — parent process ID
4. **D** — SIGINT
5. **B** — SIGTERM
6. **C** — the kernel forces immediate termination, so the process cannot catch or ignore it
7. **B** — (...) creates a subshell, { ...; } does not
8. **B** — it runs cleanup when the shell exits
9. **C** — it waits for all current background jobs
10. **D** — it bounds runtime and stops the command after 5 seconds
11. **\$!**
12. **SIGTERM**
13. **EXIT**
14. Output: **outer**
15. Output: **5**

Module 9 — Text Processing (grep / sed / awk)

Module 09 Quiz: Text Processing

Multiple Choice (10)

Choose the best answer.

1. Which grep flag makes matching case-insensitive?
A. -v
B. -i

- C. -n
 - D. -c
2. Which grep flag prints line numbers?
 - A. -l
 - B. -o
 - C. -n
 - D. -r
 3. What does grep -v pattern file do?
 - A. Shows only matching lines
 - B. Deletes matching lines from the file
 - C. Shows non-matching lines
 - D. Counts matching lines
 4. In sed 's/cat/dog/g' file, what does g mean?
 - A. Global: replace all matches on each line
 - B. Greedy matching
 - C. Group the pattern
 - D. Go to next line
 5. In sed -n 's/error/ERROR/p' file, why is p used?
 - A. To print only lines where the substitution succeeded
 - B. To pause processing
 - C. To print line numbers
 - D. To preserve the file
 6. What does \$0 mean in awk?
 - A. The first field
 - B. The entire current line
 - C. The line number
 - D. The last field
 7. What does \$NF mean in awk?
 - A. Number of files
 - B. Next field
 - C. Last field in the current record
 - D. New file
 8. What is FS in awk?
 - A. File size
 - B. Field separator used for input
 - C. Output format string
 - D. File system
 9. Why is cat file | grep word often called a UUOC?
 - A. Because grep cannot read from standard input
 - B. Because cat changes the data
 - C. Because grep word file is simpler and avoids an unnecessary cat
 - D. Because pipes are slower than files in every case
 10. Why is xargs -0 safer with filenames from find -print0?
 - A. It encrypts the filenames
 - B. It handles spaces and special characters safely using NUL separators
 - C. It sorts the filenames first
 - D. It ignores hidden files

Fill in the blanks (3)

11. In awk, ____ stores the current record number.
12. In awk, OFS stands for output ____.
13. In sed 's/foo/bar/2', the 2 means replace the _____ match on each line.

Predict the output (2)

14. What is the output of:

```
printf 'One\nTWO\nthree\n' | grep -i 'two'
```

15. What is the output of:

```
awk -F ',' 'NR > 1 {sum += $2} END {print sum}' <<'EOF'
item,qty
pen,2
book,3
bag,5
EOF
```

Answer Key

Multiple Choice

1. **B**
2. **C**
3. **C**
4. **A**
5. **A**
6. **B**
7. **C**
8. **B**
9. **C**
10. **B**

Fill in the blanks

11. **NR**
12. **field separator**
13. **second**

Predict the output

14.

TWO

15.

10

Notes

- BEGIN runs before input is read.
- END runs after all input has been processed.
- sed -n ... p is a common way to print only selected transformed lines.

Module 10 — Error Handling & Debugging

Module 10 Quiz — Error Handling & Debugging

Multiple Choice (10)

1) What does an exit status of 0 mean in bash?

- A. The command printed no output
- B. The command succeeded
- C. The command ran in the background
- D. The command returned false text

Answer: B

2) Which variable stores the exit status of the most recently executed command?

- A. \$#
- B. \$!
- C. \$?
- D. \$\$

Answer: C

3) Without set -o pipefail, what does \$? reflect after false | true?

- A. The status of the first command in the pipeline
- B. The status of the last command in the pipeline
- C. The sum of both exit codes
- D. A random non-zero code

Answer: B

4) What does set -u do?

- A. Treats unset variables as errors
- B. Disables globbing
- C. Prints executed commands
- D. Forces every command to return 0

Answer: A

5) Why is set -e considered tricky inside if conditions?

- A. It is ignored inside scripts
- B. Failing commands used as if tests are special cases and may not abort the script
- C. It only works with [[]]
- D. It converts stderr into stdout

Answer: B

6) What is the purpose of PS4 when debugging with set -x?

- A. It changes the shell prompt for interactive sessions only
- B. It stores the last exit code
- C. It customizes the prefix shown for xtrace output
- D. It disables tracing after four commands

Answer: C

7) Which pattern is best when a command failure is expected and should be handled clearly?

- A. set -e and hope for the best

- B. if ! cmd; then ... fi
- C. cmd &
- D. cmd >> /dev/null

Answer: B

8) Which statement about ERR traps is most accurate?

- A. They fire for every successful command
- B. They can centralize failure logging, but they do not fire in every context
- C. They replace the need for explicit error handling entirely
- D. They only work in POSIX sh, not bash

Answer: B

9) In a “strict mode” header, why do many scripts also set IFS=\$'\n\t'?

- A. To split words only on newlines and tabs instead of spaces too
- B. To make set -u ignore unset variables
- C. To enable arrays automatically
- D. To disable redirection

Answer: A

10) Which severity labels does shellcheck commonly use when reporting findings?

- A. low / medium / high only
- B. style / info / warning / error
- C. bronze / silver / gold
- D. note / severe / fatal

Answer: B

Fill in the Blank (3)

11) Complete the common strict-mode header:

`set -euo _____`

Answer: pipefail

12) Complete the helper:

`die() { echo "ERROR: $" "&2; exit ____; }`

Answer: 1

13) The command used to check whether curl exists in PATH is:

`_____ -v curl >/dev/null 2>&1`

Answer: command

Predict the Output / Behavior (2)

14) Predict the value of \$?

```
false | true  
echo "$?"
```

What prints if pipefail is **not** enabled?

Answer: 0

Why: Without pipefail, the pipeline status comes from the last command only, and true returns 0.

15) Predict the behavior

```
set -e  
if grep -q zebra <<< "cat dog"; then  
    echo "found"  
fi  
echo "after if"
```

What happens?

- A. The script exits before printing anything
- B. It prints found only
- C. It prints after if
- D. It prints a bash syntax error

Answer: C

Why: grep is being used as the if test. That is a special case where set -e does not automatically abort the script.

Answer Key

1. B
2. C
3. B
4. A
5. B
6. C
7. B
8. B
9. A
10. B
11. pipefail
12. 1
13. command
14. 0
15. C

Module 11 — Advanced Patterns

Module 11 Quiz — Advanced Bash

Multiple Choice (10)

1) What does the option string ":hv:f:" mean in getopts?

- A. h, v, and f all require values
- B. h and v are flags; f requires a value
- C. only v requires a value
- D. bash should ignore errors

Answer: B

2) Which variable holds the current option argument in getopt?

- A. \$1
- B. \$OPT
- C. \$OPTARG
- D. \$ARGV

Answer: C

3) Which pattern is commonly used for long options in bash?

- A. select ... in
- B. while [[\$# -gt 0]]; do case "\$1" in ... esac; done
- C. for opt in "\$@"
- D. read --long

Answer: B

4) What is the main difference between ((x = 5 + 2)) and x=\$((5 + 2))?

- A. The first is invalid bash
- B. The second supports floating point and the first does not
- C. (()) is an arithmetic command; \$(()) expands to a value
- D. They are always identical in every context

Answer: C

5) What happens with a quoted here-doc delimiter like <<'EOF'?

- A. Variables are expanded twice
- B. Variable interpolation is disabled
- C. Only command substitution is disabled
- D. The file becomes executable

Answer: B

6) Why does let "x = 5 / 2" assign 2 to x?

- A. let rounds to nearest integer
- B. let uses integer arithmetic in bash
- C. / means modulo in let
- D. because quotes force string math

Answer: B

7) What does source file.sh do?

- A. Runs the script in a new shell process
- B. Loads and executes the file in the current shell
- C. Compiles the script
- D. Makes the file executable

Answer: B

8) In [["\${BASH_SOURCE[0]}" == "\${0}"]], what is this check usually for?

- A. Detecting whether a file was executed directly or sourced

- B. Checking whether bash exists
- C. Verifying the user is root
- D. Comparing two filenames alphabetically

Answer: A

9) Which command creates a named pipe?

- A. mkpipe
- B. pipe
- C. mkfifo
- D. coproc

Answer: C

10) Which tool is commonly used for floating-point math in shell scripts?

- A. bc or awk
- B. sort
- C. chmod
- D. mkfifo

Answer: A

Fill in the Blank (3)

11) The shorthand POSIX-style command that behaves like `source file.sh` is _____ `file.sh`.

Answer: .

12) In bash, `(( ))` and `${(( ))}` both perform _____ arithmetic.

Answer: integer

13) The `getopts` variable that tracks the next argument index is _____.

Answer: OPTIND

Predict the Output / Behavior (2)

14) Predict the output:

```
x=3
echo $(( x * 4 ))
```

Answer: 12

15) Predict the output:

```
name="Bash"
cat <<'EOF'
Hello $name
EOF
```

Answer: Hello \$name

